



ViajaYa

Sistema de Reservaciones para Agencia de Viajes

ENTREGABLE 6

Documentación Básica del Sistema

Documentación generada con IA Claude en base a información ya existente y verificada

Proyecto de Programación Orientada a Objetos en Java

Universidad de Cartagena

2025 – 2026

1. Descripción General del Sistema

ViajaYa es una aplicación de escritorio desarrollada en Java que permite gestionar el proceso de reservas de vuelos para una agencia de viajes. El sistema está orientado a usuarios finales (clientes) que desean consultar vuelos disponibles, crear reservas, gestionar sus itinerarios y administrar su cuenta de manera autónoma.

La aplicación implementa los principios de la Programación Orientada a Objetos (herencia, encapsulamiento, polimorfismo y abstracción) y ofrece una interfaz gráfica construida con Swing, con persistencia de datos mediante serialización binaria de objetos Java.

El sistema cubre el ciclo completo de interacción del usuario con la agencia:

- Registro e inicio de sesión con validación de credenciales.
- Consulta de vuelos con filtros por horario, tarifa, aerolínea, fecha y preferencia de vuelo directo.
- Creación de reservas con itinerarios compuestos por uno o varios vuelos.
- Registro de pasajeros de distintos tipos (Adulto, Niño, Adulto Mayor) con sus datos y validaciones propias.
- Asignación de asientos por categoría (Económica, Ejecutiva, Primera Clase) con cálculo de tarifas de referencia.
- Consulta, visualización y cancelación de reservas activas.
- Administración del perfil del usuario (actualizar datos o cancelar cuenta).
- Generación automática de vuelos cuando no existen resultados para una búsqueda.

El sistema no procesa pagos reales ni transacciones bancarias; los valores mostrados son exclusivamente informativos para efectos académicos.

2. Requisitos Funcionales Implementados

ID	Requisito funcional implementado	Estado
RF-01	Registro de usuario con validación de login único, correo con formato válido y contraseña confirmada.	☒ Completo
RF-02	Inicio de sesión con validación de login y contraseña.	☒ Completo
RF-03	Cierre de sesión y retorno a la pantalla de bienvenida.	☒ Completo
RF-04	Consulta de vuelos por horario con filtros (origen, destino, fecha, aerolínea, directos/con escala).	☒ Completo
RF-05	Consulta de vuelos ordenados por tarifa de referencia (menor a mayor).	☒ Completo
RF-06	Visualización del estado del vuelo seleccionado (programado, en hora, retrasado, finalizado, cancelado) y disponibilidad de asientos.	☒ Completo
RF-07	Generación automática de vuelos cuando no existen resultados para el origen/destino/fecha indicados.	☒ Completo
RF-08	Creación de reserva con itinerario de uno o varios vuelos.	☒ Completo
RF-09	Registro de pasajeros de tipo Adulto, Niño y Adulto Mayor con validaciones propias por tipo.	☒ Completo
RF-10	Asignación de asientos disponibles por categoría (Económica, Ejecutiva, Primera Clase) a cada pasajero.	☒ Completo
RF-11	Cálculo del valor informativo total de la reserva incluyendo recargos por categoría de asiento.	☒ Completo
RF-12	Confirmación de reserva con validación de itinerario y pasajeros no vacíos.	☒ Completo
RF-13	Consulta y visualización del detalle de reservas activas del usuario.	☒ Completo
RF-14	Cancelación de reservas con liberación automática de los asientos asociados.	☒ Completo
RF-15	Actualización de datos del perfil de usuario (nombres, correo, contraseña).	☒ Completo
RF-16	Cancelación del registro de usuario con cancelación previa de sus reservas activas.	☒ Completo
RF-17	Persistencia de datos mediante serialización binaria; carga automática al iniciar y guardado al salir o confirmar operaciones.	☒ Completo

3. Explicación de las Clases Principales

Clase	Descripción y responsabilidades principales
AgenciaViajes	Clase fachada del sistema. Administra las listas de usuarios, aerolíneas, vuelos y reservas. Expone todos los métodos de negocio que usa la interfaz gráfica: registrar/iniciar sesión, consultar/filtrar vuelos, crear/cancelar reservas.
Usuario	Representa un cliente registrado. Almacena id, nombres, email, login, contraseña y la lista de reservas asociadas. Ofrece validación de credenciales y consulta de reservas activas.
Vuelo (abstracta)	Clase base para todos los vuelos. Define atributos comunes (id, origen, destino, fecha, horarios, tarifa base, estado, aerolínea, asientos, ciudades de escala) y el método abstracto calcularTarifaFinal(), que cada subclase implementa con su propia lógica de impuestos.
VueloNacional	Extiende Vuelo. Aplica un impuesto nacional (8–10%) sobre la tarifa base para calcular la tarifa final. Representa vuelos dentro de Colombia.
VueloInternacional	Extiende Vuelo. Aplica un impuesto internacional (12–20%) sobre la tarifa base. Representa vuelos a destinos en el extranjero, con soporte para ciudades de escala.
Asiento (abstracta)	Clase base para los asientos de un vuelo. Define número, categoría y disponibilidad; incluye métodos reservar() y liberar(). Declara los métodos abstractos getCategoriaNombre(), getServicios() y getFactorPrecio() para el polimorfismo de categorías.
Economico	Extiende Asiento. Factor de precio 1.0 (sin recargo). Servicio: 1 pieza de equipaje de mano hasta 8 kg.
Ejecutivo	Extiende Asiento. Factor de precio 1.8 (80% de recargo). Servicio: acceso a sala VIP + 2 piezas de equipaje hasta 23 kg.
PrimeraClase	Extiende Asiento. Factor de precio 3.0 (200% de recargo). Servicio: menú gourmet + sala VIP + 3 piezas de equipaje.
Pasajero (abstracta)	Clase base para los pasajeros de una reserva. Almacena id, nombre, edad y contacto. Declara los métodos abstractos validarDatos() y getTipo() para polimorfismo.
Adulto	Extiende Pasajero. Valida que el pasajero tenga ≥ 18 años y datos básicos completos.
Nino	Extiende Pasajero. Almacena el nombre del acompañante adulto. Valida edad 0–17 años y que el acompañante esté registrado.
AdultoMayor	Extiende Pasajero. Almacena si requiere asistencia especial. Valida edad ≥ 60 años.
Reserva	Agrupar un usuario, un itinerario, una lista de pasajeros y el mapa de asientos asignados. Gestiona la asignación (y reasignación) de asientos, calcula el total informativo y permite confirmar o cancelar la reserva liberando los asientos.
Itinerario	Lista ordenada de vuelos que conforman el plan de viaje de una reserva. Calcula duración total y costo acumulado.

Clase	Descripción y responsabilidades principales
Aerolinea	Representa una aerolínea con nombre y código IATA. Mantiene la lista de vuelos que opera.
GeneradorVuelos	Clase utilitaria. Genera vuelos aleatorios pero realistas cuando no hay resultados en una búsqueda; también crea y distribuye asientos por sección (Primera Clase, Ejecutiva, Económica) y asigna ocupación simulada.
CatalogoDatos	Clase de constantes de referencia: ciudades nacionales/internacionales, aerolíneas, rangos de tarifas, duraciones y ciudades de escala. No tiene lógica de negocio ni persistencia propia.
GestorPersistencia	Gestiona la serialización y deserialización del objeto AgenciaViajes a/desde el archivo binario data/agencia.dat. Crea datos de demo si el archivo no existe.

4. Relaciones Más Importantes del Diagrama UML

Clases involucradas	Tipo de relación	Descripción
AgenciaViajes → Usuario AgenciaViajes → Vuelo AgenciaViajes → Aerolinea AgenciaViajes → Reserva	Composición	AgenciaViajes posee y gestiona las listas de usuarios, vuelos, aerolíneas y reservas. Si la agencia se elimina, todos estos objetos dejan de existir con ella.
Vuelo ← VueloNacional Vuelo ← VueloInternacional	Herencia	Ambas clases concretas especializan Vuelo (clase abstracta), sobrescribiendo calcularTarifaFinal() con su propia lógica de impuestos.
Asiento ← Economico Asiento ← Ejecutivo Asiento ← PrimeraClase	Herencia	Las tres categorías concretas extienden Asiento (clase abstracta), implementando getCategoriaNombre(), getServicios() y getFactorPrecio(), lo que permite polimorfismo en el cálculo de precios.
Pasajero ← Adulto Pasajero ← Nino Pasajero ← AdultoMayor	Herencia	Los tres tipos de pasajero concretos extienden Pasajero (clase abstracta), implementando validarDatos() con reglas propias por tipo de pasajero.
Vuelo → Asiento	Composición	Un vuelo contiene una lista de asientos que le pertenecen. Los asientos no existen fuera del contexto del vuelo al que pertenecen.
Reserva → Itinerario	Composición	Cada reserva posee exactamente un itinerario que reúne los vuelos seleccionados.
Reserva → Pasajero Reserva → Asiento	Asociación	Una reserva agrupa una lista de pasajeros y mantiene un mapa que relaciona el id de cada pasajero con su asiento asignado.
Usuario → Reserva	Asociación	Un usuario puede tener múltiples reservas. La reserva referencia de vuelta al usuario al que pertenece.
Aerolinea → Vuelo	Asociación	Una aerolínea opera una lista de vuelos. Cada vuelo referencia a la aerolínea que lo ofrece.
Itinerario → Vuelo	Asociación	El itinerario mantiene una lista ordenada de vuelos que componen el plan de viaje.

5. Descripción de la Interfaz Gráfica

La interfaz gráfica fue construida con Java Swing siguiendo una identidad visual coherente definida en la clase Estilos. La navegación entre pantallas se implementa mediante CardLayout sobre un JPanel contenedor en la VentanaPrincipal.

Panel / Pantalla	Descripción y componentes principales
PanelBienvenida	Pantalla de inicio con degradado azul marino/azul cielo, el nombre de la agencia, un listado de los servicios ofrecidos y botones para Iniciar Sesión o Registrarse.
PanelLogin	Formulario de inicio de sesión con campos de usuario y contraseña, validación de campos vacíos y mensajes de error ante credenciales inválidas.
PanelRegistro	Formulario de registro con campos de nombre, correo, login y contraseña (con confirmación). Incluye validación de formato de correo, contraseñas coincidentes y login único.
PanelMenu	Menú principal con cuatro botones grandes con iconos: Consultar Vuelos, Crear Reserva, Mis Reservas y Mi Cuenta. Muestra un mensaje de bienvenida personalizado con el nombre del usuario.
PanelConsultaVuelos	Panel de búsqueda con combos de origen/destino, selector de fecha (con calendario personalizado), campo de aerolínea opcional, checkbox de vuelos directos y orden por horario o tarifa. Muestra resultados en una tabla y el estado del vuelo seleccionado en un área de texto inferior.
PanelReserva	Pantalla de creación de reservas dividida en tres pestañas: (1) Itinerario: agregar/quitar vuelos por ID. (2) Pasajeros: formulario con tipo de pasajero (Adulto/Niño/Adulto Mayor) y campos dinámicos según el tipo. (3) Asientos: selección de vuelo, pasajero, categoría y asiento disponible con recargo calculado. Muestra el total informativo en tiempo real.
PanelMisReservas	Tabla con las reservas del usuario (id, fecha, estado, vuelos, pasajeros, total). Panel inferior con detalle de la reserva seleccionada (itinerario, pasajeros, asientos). Botones para ver detalle y cancelar reserva.
PanelCuenta	Formulario para actualizar nombres, correo y contraseña del usuario actual. Incluye opción de cancelar el registro con confirmación previa.
Estilos	Clase utilitaria de diseño visual. Define la paleta corporativa (azul marino, azul cielo, dorado, grises) y ofrece métodos fábrica para crear componentes estilizados de manera consistente (botones, etiquetas, campos, tablas, combos, headers con degradado).
VentanaPrincipal	Ventana raíz (JFrame) que administra la navegación entre pantallas mediante CardLayout. Maneja el ciclo de vida de la aplicación: guarda los datos al confirmar operaciones o al cerrar la ventana (con diálogo de confirmación).

5.1 Paleta de colores y fuentes

La paleta corporativa de ViajaYa utiliza los siguientes colores y tipografías:

- Azul marino (#0B1F3A): color principal de encabezados y fondos de tablas.
- Azul cielo (#2E86DE): color de acento para botones secundarios y bordes activos.
- Dorado (#F0A500): color de botones de acción principal y elementos destacados.
- Fondo claro (#F8FAFF) y fondo panel (#E8EDF5): fondos generales y de paneles secundarios.
- Tipografía SansSerif en tamaños 12pt (normal), 14pt (énfasis) y 18–26pt (títulos).

6. Explicación del Mecanismo de Persistencia Mediante Archivos

El sistema implementa persistencia mediante serialización de objetos Java (`java.io.ObjectOutputStream` / `ObjectInputStream`). El estado completo de la agencia se guarda en un único archivo binario:

6.1 Clase responsable: `GestorPersistencia`

La clase `GestorPersistencia` (paquete `com.agenciaviajes.persistencia`) centraliza toda la lógica de lectura y escritura. Ofrece dos métodos estáticos:

- `guardar(AgenciaViajes agencia)`: serializa el objeto `AgenciaViajes` completo al archivo `data/agencia.dat` usando `ObjectOutputStream`. Crea el directorio `data/` si no existe.
- `cargar(String nombreAgencia)`: deserializa el objeto desde `data/agencia.dat` con `ObjectInputStream`. Si el archivo no existe o hay error, crea una nueva instancia y carga datos de demostración (`cargarDatosDemo()`).

6.2 Ruta y formato del archivo

- Ruta relativa al directorio de ejecución: `data/agencia.dat`
- Formato: binario (Java Object Serialization, `.dat`).
- Contenido serializado: el grafo completo de objetos — usuarios, aerolíneas, vuelos, asientos, reservas e itinerarios.

6.3 Cuándo se guarda

- Al registrar un nuevo usuario exitosamente.
- Al actualizar o cancelar el perfil del usuario.
- Al confirmar una reserva.
- Al cerrar la ventana principal (con diálogo de confirmación).
- Al cerrar sesión.

6.4 Serialización de las clases

Todas las clases del modelo implementan `java.io.Serializable` e incluyen un campo `serialVersionUID` para garantizar la compatibilidad entre versiones. Al cargar los datos, el método `repararVuelosSinAsientos()` reconstruye los asientos de vuelos que hayan quedado con listas vacías por alguna inconsistencia en el archivo.

7. Dificultades Encontradas Durante el Desarrollo

7.1 Diseño de la jerarquía de clases

La principal dificultad inicial fue definir correctamente qué clases debían ser abstractas y cuáles concretas. Para Vuelo, Asiento y Pasajero no tenía sentido instanciar la clase base directamente; se identificó la necesidad de métodos abstractos (`calcularTarifaFinal`, `getServicios`, `validarDatos`) que cada subclase implementa con su propia lógica, materializando el polimorfismo.

7.2 Generación de vuelos realistas

Se requería que la aplicación tuviese vuelos disponibles desde el inicio y que generara nuevos cuando el usuario buscara combinaciones no existentes. La solución fue la clase `GeneradorVuelos`, que usa `CatalogoDatos` para obtener tarifas, duraciones y aerolíneas coherentes según la distancia y el tipo de vuelo (nacional/internacional). El ajuste de los factores de demanda para la ocupación aleatoria de asientos requirió varias iteraciones.

7.3 Integridad de los asientos en la reserva

Garantizar que un mismo asiento no fuera asignado a dos pasajeros distintos y que al cancelar una reserva los asientos quedaran libres correctamente fue un punto crítico. Se implementó el mapa `asientosAsignados` (`idPasajero` → `Asiento`) en `Reserva` para controlar estas restricciones, y el método `cancelar()` itera sobre el mapa para liberar cada asiento.

7.4 Persistencia y reparación de datos

Al cargar el archivo serializado se detectó que algunos vuelos podían quedar sin asientos (listas vacías) si el archivo era de una versión anterior o había sido generado de manera incompleta. La solución fue el método `repararVuelosSinAsientos()` en `AgenciaViajes`, que regenera los asientos de cualquier vuelo que los tenga vacíos al arrancar.

7.5 Selector de fechas personalizado

Java Swing no incluye un componente nativo de calendario. Se construyó un selector de fechas propio (en `PanelConsultaVuelos`) usando un `JDialog` no decorado con un `GridLayout` de botones de días, con lógica para respetar el mes/año seleccionado y bloquear fechas inválidas según el tipo de selección (ida o vuelta).

7.6 Navegación sin pérdida de estado

Gestionar el paso de datos entre paneles (por ejemplo, el ID del vuelo seleccionado en `Consulta de Vuelos` para precargar el campo en `Crear Reserva`) sin violar la encapsulación requirió el patrón de preselección: `PanelReserva` expone el método `preseleccionarVuelo(idVuelo)`, que `VentanaPrincipal` llama antes de mostrar el panel.

8. Aportes Recibidos de la IA Generativa y Forma en que Fueron Validados

Durante el desarrollo del proyecto se utilizaron herramientas de inteligencia artificial generativa (Claude de Anthropic) para apoyar el análisis de diseño, sugerir estructuras de clases, revisar lógica de negocio y detectar posibles inconsistencias. A continuación se describe cómo se validaron y adaptaron esos aportes:

Aporte de la IA	Forma en que fue validado	Ajuste realizado por el equipo
Propuesta de jerarquía de clases (Vuelo abstracto, VueloNacional, VueloInternacional).	Se comparó con el análisis propio del enunciado. Se comprobó que el polimorfismo de calcularTarifaFinal() era coherente con los requisitos.	Se ajustaron los porcentajes de impuesto para que reflejaran rangos realistas del mercado colombiano (8–10% nacional, 12–20% internacional).
Estructura del CatalogoDatos con mapas de ciudades y rangos de tarifas por categoría.	Se verificó que las ciudades y aerolíneas listadas correspondieran a operaciones reales en Colombia.	Se añadieron ciudades colombianas adicionales y se ajustaron los rangos de tarifas basándose en precios de referencia del mercado aéreo colombiano.
Sugerencia de usar HashMap<String, Asiento> en Reserva para mapear pasajero→asiento.	Se probó el flujo completo de asignación, reasignación y cancelación para verificar que los asientos se liberaban correctamente.	Se implementó el método reasignarAsientoAPasajero() para contemplar el caso en que el usuario cambia de asiento sin liberar el anterior de forma incorrecta.
Implementación del GestorPersistencia con ObjectOutputStream.	Se realizaron pruebas de cierre y reapertura de la aplicación verificando que los datos (usuarios, reservas, vuelos generados) persistían correctamente.	Se añadió el método repararVuelosSinAsientos() para solucionar inconsistencias detectadas en archivos .dat de versiones anteriores.
Esquema del CardLayout para la navegación entre paneles en VentanaPrincipal.	Se navegó entre todas las pantallas en distintos escenarios (sesión activa, sin sesión, con reserva en progreso) para verificar que no hubiera estados inconsistentes.	Se añadió el mecanismo de preseleccionarVuelo() y el método actualizar() en cada panel para refrescar datos al mostrarse.
Diseño del panel de reserva con pestañas (itinerario, pasajeros, asientos).	Se probaron flujos completos: agregar vuelo → agregar pasajero → asignar asiento → confirmar reserva, verificando que el total se calculara correctamente.	Se añadió la lógica de descarte de reserva en progreso al volver al menú, para evitar reservas vacías guardadas en la lista del usuario.

En todos los casos, el equipo revisó críticamente las sugerencias, realizó pruebas de funcionamiento y adaptó el código a las necesidades específicas del proyecto antes de incorporarlo al sistema. La IA fue empleada como herramienta de apoyo para pensar y verificar, no como reemplazo del análisis propio del equipo.

9. Estructura de Paquetes del Proyecto

El proyecto se organiza en cuatro paquetes Java bajo el paquete raíz `com.agenciaviajes`:

- `com.agenciaviajes` — Clase `App` (punto de entrada de la aplicación).
- `com.agenciaviajes.modelo` — Todas las clases del dominio: `AgenciaViajes`, `Usuario`, `Vuelo`, `VueloNacional`, `VueloInternacional`, `Asiento`, `Economico`, `Ejecutivo`, `PrimeraClase`, `Pasajero`, `Adulto`, `Nino`, `AdultoMayor`, `Reserva`, `Itinerario`, `Aerolinea`, `GeneradorVuelos`, `CatalogoDatos`.
- `com.agenciaviajes.gui` — Todos los paneles Swing: `VentanaPrincipal`, `Estilos`, `PanelBienvenida`, `PanelLogin`, `PanelRegistro`, `PanelMenu`, `PanelConsultaVuelos`, `PanelReserva`, `PanelMisReservas`, `PanelCuenta`.
- `com.agenciaviajes.persistencia` — `GestorPersistencia`.

El archivo de datos binario se almacena en: `data/agencia.dat` (relativo al directorio de ejecución del proyecto).